

MÓDULO PROYECTO INTERMODULAR

Ciclo Superior Desarrollo de Aplicaciones Web

Departamento: Informática y Comunicaciones

IES “María Moliner”

Curso: 2025/2026

Grupo: S2I

Proyecto: MrFinance

Carlos Javier Pilar Sanz

Email: carlosj.pilsan.1@educa.jcyl.es

Tutor individual: María José

Tutor colectivo: Enrique Carballo Albarrán

Fecha de presentación: 4 Junio 2026



Este trabajo se distribuye bajo licencia Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional.

" "

"

Doy gracias por el apoyo en el desarrollo de este proyecto a:

" " " " "

" " " " " " "María José"

" " " " " " " " "

" " " "

Iván Santos Diez

Diego de la Esperanza

Ismael Monjas

Carlos Mesonero

Hugo San Juan Morales

Javier Bartolome

Ashkan Seminov Asimov

" " " " " " " " "

" " " " " "

" "

"

"

"

" " "

" " "

"

" "

"

"

" "

" " " "

" "

"

" "

" "

" " "

"

" "

"

" "

"

"

"

"

"

"

" "

" "

" "

"

"

" "

"

" "

" "

" "

" "

" "

"

"

"

" "

"

"

" "

"

" "

" "

"

"

"

"

"

"

"

"

"

" "

" "

"

" "

"

" "

"

"

" "

"

" "

"

" "

" "

"

"

"

" "

"

"

" "

"

" "

" "

" "

" "

"

" "

" "

"

"

"

" "

"

" "

" "

" "

" "

" "

" "

" "

" "

" "

"

" "

" "

" "

" "

" "

" "

" "

"

"

"

" "

" "

" "

" "

" "

" "

" "

"

"

"

" "

" "

" "

" "

" "

" "

" "

"

"

"

" "

" "

"

"

" "

" "

" "

" "

" "

" "

" "

" "

" "

" "

"

" "

" "

" "

" "

" "

" "

" "

" "

"

" "

" "

" "

" "

" "

" "

" "

"

"

"

" "

"

" "

" "

"

" "

" "

" "

" "

"

" "

" "

" "

" "

" "

" "

" "

" "

" "

" "

" "

" "

" "

" "

" "

"

"

"		
" "		2
" "	" "	2
"		3
"	"	4
"	"	5
" "		9
" "		9
" "	" "	10
" "	" "	10
" "	" "	10
" "	" "	12
" "	" "	12
" "	" "	13
" "	" "	15
" "	" "	15
" "	" "	16
" "	" "	16
" "	" "	18
" "	" "	18
" "	" "	18
" "	" "	19
" "	" "	19
" "	" "	19
" "	" "	29
" "	" "	29
" "	" "	29
" "	" "	31
" "	" "	31
" "	" "	31

"	31
"	31
" " " "	32
"	32
" " " "	32
" "	33
" " " "	33
"	33
	34
Como se a orientado finalmente la arquitectura en la versión de Docker.	36
	37
"	37
" " " " "	39
	40
" "	40
Cuestiones de implementación y codificación reseñables	42
6.5 Gestión de y versionado	49
6.6 Pruebas realizadas	51
7. Manuales de usuario	54
7.1 Manual de usuario y configuración	54
7.2 Manual de instalación y configuración.	54
8. Posibles ampliaciones	56
9. Conclusiones y valoraciones	57
10. Bibliografía /Webgrafía	58
11. Anexos	59
"	
" "	

11

11

11

"

11

11

"

11

11

11

11

11

11

11

11

11

11

11

[illegible]

" " " "

"

¿Cuál es el problema real?

" " " una visión clara de su situación económica" " gastos se dispersan" " " " "

" " " " " " " "

" " " " " " " "

" " " " " "

- " Falta de visibilidad " " " " " " " "
- " Toma de decisiones activa " " " " " " " "
- " Dispersión de información " " " " " " " "

Subproblema	Consecuencia actual	Solución que aporta MrFinance
Registro manual y disperso de gastos/ingresos	" " " "	" " " "
Facturas en papel o perdidas	" " " "	" " "
Sin estadísticas automáticas	" " " "	" " "
Incapacidad de anticipar gastos futuros	" " " "	" " " "
Acceso no seguro a datos sensibles	" " " "	" " " "

" " " " "

Lo que Sí resuelve:"

- " " " " " " " "
- " " " " " " " "
- " " " " " " " "
- " " " " " " " "

- " " " " "

Lo que NO aborda"(Posibles Mejoras a futuro en producción):"

- " " " " " " " "
- " " " " " " " "
- " " " "
- " " " " " " " "

Justificación técnica

" " " " " full-stack" " conocimientos de ciberseguridad" "

- " " " " " " " "base de datos + backend con autenticación"
- " " " " " "segura" "resistente a ataques "
- " " " " " " " "frontend dinámico con gráficos"
- " " " " " " " "gestión de ficheros en servidor"
- " " " " " " " "diseño responsive + despliegue en cloud."
- "

Resumen de la propuesta:

MrFinance" " " " " " " " " " " " " " " " "

"

"

"

"

"

"

"

"

" " " " "

"

" " " " " " " " " "

" " " " " " " " " "

" " " " " " " " " "

"

•" "capítulo 0" " " " " " " " "

" " " " " " " " " "

" " " " "

"

•" "capítulo 1" " " " " " " " "

" " " " " " " " " "

" " " " " " " " " "

" " " " " " " "

"

•" "capítulo 2" " " " " " " " "

" " " " " " "

"

•" "capítulo 3" " " " " " " " "

" " " " " " " " " "

" " " " " " " " " "

"

•" "capítulo 4" " " " " " " " "

" " " " " " " " " "

" " " " " " " " " "

" " " " " " " " " "

" " " " " " " " " "

"

•" "capítulo 5" " " " " " " " "

" " " " " " " " " "

" " " " " "

"

•" "capítulo 6" " " " " " " " "

" " " " " " " " " "

" " " " " " " " " "

" " " " " " " " " "

[illegible]

"Scrum "

'visual studio code y nano

" vite, nodejs, react, javascript, html, css,

"MYSQL.

"diagrams.io "Google Stich y

|| || || || || ||

"Docker.



" "

"git" "github."



!!

|| || || || || || ||

"nginx"

```
certbot "lets"encrypt "freemyip" "wireguard"
```



"

"

"

"

"

"

"

"

"

"

" " " " una aplicación web de gestión de ahorros
personal " " "interfaz moderna y atractiva " " " " "
" " " " " " " " " " "
"

" " " " " " " " " " " " " "
" " " " " " " " " " " " " "
" " " " " " " " " " " " " "
" " " " " " " " " " " " " "
" " " " " " " " " " " " "

" " " " " " " " " " " " " "
" " " " " " " " " " " " " "
" " " " " " " " " " " " " "
" " " " " " " " " " " " " "

" " " " " " " " " " " " " "
" " " " " " " " " " " " " "
" " " " " " " " " " " " "

"

"

" "

"

"

"

"

"

•

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

•

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

•

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

•

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"


```

1  services:
2
3  # -----
4  # SERVICIO 1: Node.js (node:20-alpine)
5  # -----
6  >Run Service
7  app:
8    #build . apra usar el Dockerfile
9    build: .
10   container_name: nodejs_app
11   restart: no
12   ports:
13     - "3000:3000"
14   volumes:
15     - ./app:/app
16   environment:
17     - NODE_ENV=development
18     - DB_HOST=db
19     - DB_PORT=3306
20     - DB_USER=user
21     - DB_PASSWORD=user
22     - DB_NAME=appdb
23   depends_on:
24     db:
25       condition: service_healthy
26
27 # -----
28 # SERVICIO 2: Base de datos MySQL
29 # -----
30 >Run Service
31 db:
32   # Usa la imagen oficial de MySQL 8.0 directamente desde Docker Hub.
33   image: mysql:8.0
34   #nombre del contenedor
35   container_name: mysql_db
36   #restart no apra que no se ejecute todo el rato
37   restart: no
38   # Expone el puerto de MySQL a tu máquina local.
39   # Útil para conectarte con clientes como workbench
40   ports:
41     - "3306:3306"
42
43   # Variables de configuración de MySQL.
44   environment:
45     - MYSQL_ROOT_PASSWORD=root          # Contraseña del usuario root (admin total)
46     - MYSQL_DATABASE=appdb              # Crea esta base de datos automáticamente
47     - MYSQL_USER=user                   # Crea este usuario con acceso a appdb
48     - MYSQL_PASSWORD=user               # Contraseña del usuario appuser
49
50   volumes:
51     # Volumen named: guarda los datos de MySQL en un volumen gestionado
52     # por Docker. Los datos persisten aunque pares o borres el contenedor.
53     - mysql_data:/var/lib/mysql

```

```

52
53     # Archivo SQL opcional de inicialización.
54     # Si existe ./init.sql, MySQL lo ejecuta la PRIMERA vez que arranca
55     - ./init.sql:/docker-entrypoint-initdb.d/init.sql
56
57     # Healthcheck: Docker ejecuta este comando periódicamente para saber
58     # si MySQL está listo para recibir conexiones.
59     # El servicio "app" no arrancará hasta que esto devuelva éxito.
60     healthcheck:
61         test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "user", "-puser"]
62         interval: 1s # Comprueba cada 1 segundos
63         timeout: 5s  # Si no responde en 5s, cuenta como fallo
64         retries: 5    # Tras 5 fallos seguidos, el servicio se marca como unhealthy
65
66     # -----
67     # SERVICIO 3: phpMyAdmin
68     # -----
69     >Run Service
70     phpmyadmin:
71         # Imagen oficial de phpMyAdmin.
72         image: phpmyadmin:latest
73         #nombre del contenedor
74         container_name: phpmyadmin
75         #restart no apra que no se ejecute todo el rato
76         restart: no
77         # Puerto de acceso: en tu navegador.
78         ports:
79             - "8080:80"
80         #ajustes de php my admin
81         environment:
82             - PMA_HOST=db
83             - PMA_PORT=3306.
84             - PMA_USER=user
85             - PMA_PASSWORD=user
86             - UPLOAD_LIMIT=64M
87
88         # phpMyAdmin necesita que MySQL esté funcionando antes de arrancar, lo comprueba.
89         depends_on:
90             db:
91                 condition: service_healthy
92
93     # -----
94     # VOLÚMENES nombrados
95     # -----
96     volumes:
97         mysql_data:

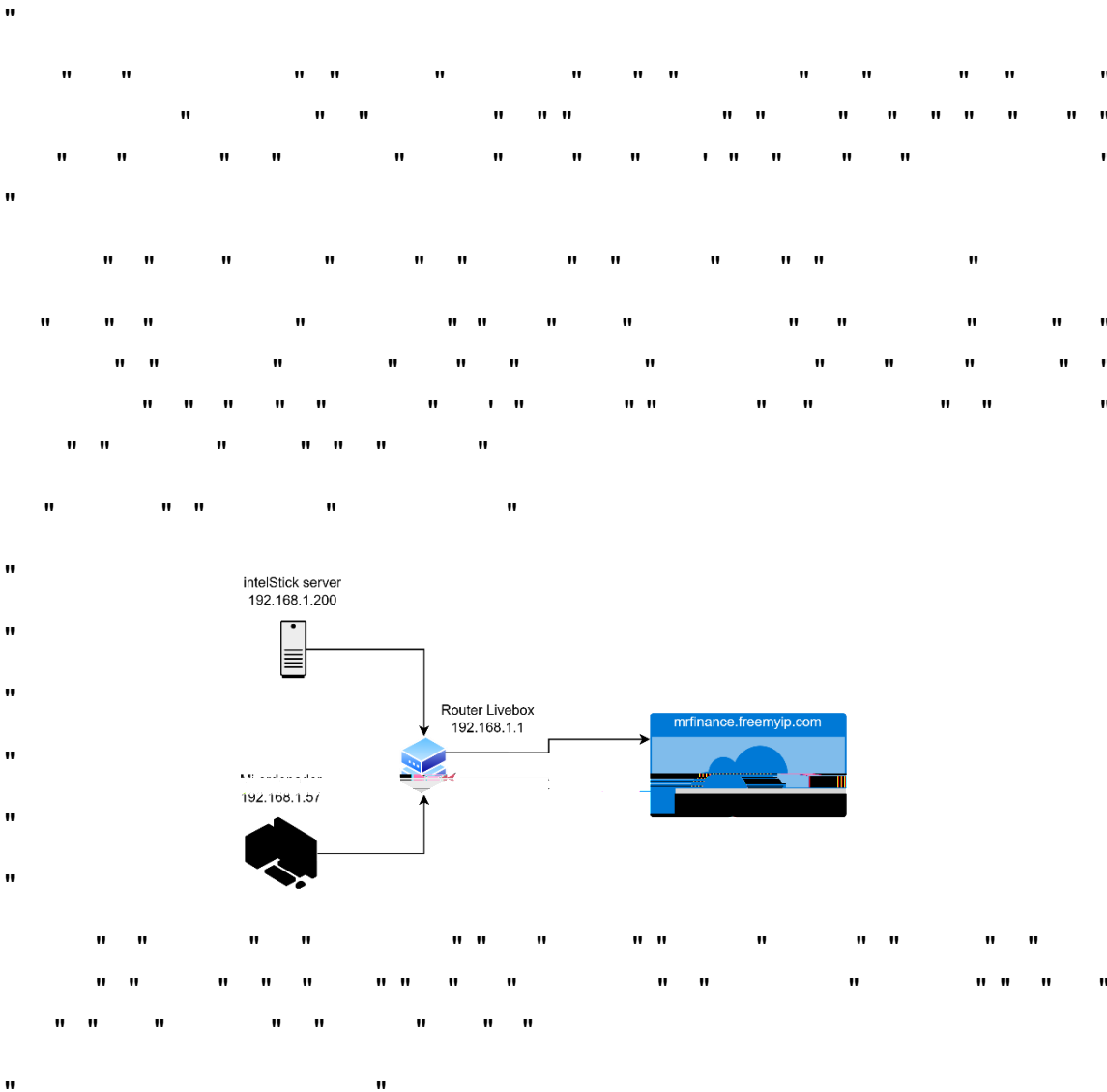
```

```

1  FROM node:20-alpine
2
3  # Directorio de trabajo
4  WORKDIR /app
5
6  # Copia los archivos de dependencias primero (aprovecha la caché de Docker)
7  # Si no existen aún, el COPY falla silenciosamente gracias al entrypoint
8  COPY entrypoint.sh /entrypoint.sh
9  RUN chmod +x /entrypoint.sh
10
11  ENTRYPOINT ["/entrypoint.sh"]

```

```
1  #!/bin/sh
2  set -e
3
4  echo "Iniciando contenedor Node.js..."
5
6  # Si existe package.json, instala dependencias y arranca
7  if [ -f "/app/package.json" ]; then
8      echo "package.json detectado. Instalando dependencias..."
9      cd /app
10     npm install
11
12     # Arranca con el script "start" si existe, si no con node directamente
13     if npm run | grep -q "^*start$"; then
14         echo "Ejecutando: npm start"
15         exec npm start
16     else
17         # Intenta adivinar el entry point
18         ENTRY=$(node -e "const p=require('/package.json'); console.log(p.main||'/index.js');">/dev/null || echo "/index.js")
19         echo "Entrypoint: $ENTRY"
20         exec node "$ENTRY"
21     fi
22 else
23     echo "No se encontró /app/package.json"
24     echo "  Monta tu proyecto en ./app y reinicia el contenedor."
25     echo "  > Modifica el container action para poder debuggear"
26     exec tail -f /dev/null
27 fi
```



[illegible]

```

4 # — Node.js + vite
5 ▷ Run Service
6 app:
7   build:
8     context: .
9     dockerfile: Dockerfile
10  container_name: node_app2
11  volumes:
12    - ./app/app # código fuente en ./app del host
13    - mrfinance_root_modules:/app/mrfinance/node_modules
14    - mrfinance_server_modules:/app/mrfinance/server/node_modules
15    - mrfinance_client_modules:/app/mrfinance/client/node_modules
16  ports:
17    - "587:587"
18    - "3000:3000"
19    - "5173:5173"
20    - "8081:8081"
21    # - "0.0.0.0:443:5173"
22    # - "0.0.0.0:80:5173"
23  environment:
24    NODE_ENV: development
25    DB_HOST: db
26    DB_PORT: 3306
27    DB_USER: user
28    DB_PASSWORD: user
29  depends_on:
30    db:
31      condition: service_healthy
32  networks:
33    - dev_net
34  restart: no

```

```
# — Nginx
> Run Service
nginx:
  image: jonasal/nginx-certbot:latest
  container_name: nginx_server
  ports:
    - "80:80"
    - "443:443"
  environment:
    - CERTBOT_EMAIL=noreply.mrfinance@gmail.com
  volumes:
    - ./app/mrfinance/client/dist:/usr/share/nginx/html:ro
    - ./nginx.conf:/etc/nginx/user_conf.d/default.conf:ro
    - certbot_letsencrypt:/etc/letsencrypt
  depends_on:
    - app
  networks:
    - dev_net
  restart: no
```

```
# — MySQL —
▷ Run Service
db:
  image: mysql:latest
  container_name: mysql2
  environment:
    MYSQL_ROOT_PASSWORD: root
    MYSQL_DATABASE: MrFinanceV2
    MYSQL_USER: user
    MYSQL_PASSWORD: user
  volumes:
    - mysql_data:/var/lib/mysql
    - ./db/init:/docker-entrypoint-initdb.d
  ports:
    - "3306:3306"
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-proot"]
    interval: 10s
    timeout: 5s
    retries: 5
  networks:
    - dev_net
  restart: no
```

```
# — phpMyAdmin —
▷ Run Service
phpmyadmin:
  image: phpmyadmin:latest
  container_name: phpmyAdmin2
  environment:
    PMA_HOST: db
    PMA_PORT: 3306
    PMA_USER: root
    PMA_PASSWORD: root
    UPLOAD_LIMIT: 100M
  ports:
    - "8080:80"
  depends_on:
    db:
      condition: service_healthy
  networks:
    - dev_net
  restart: no
```

" " " " " "

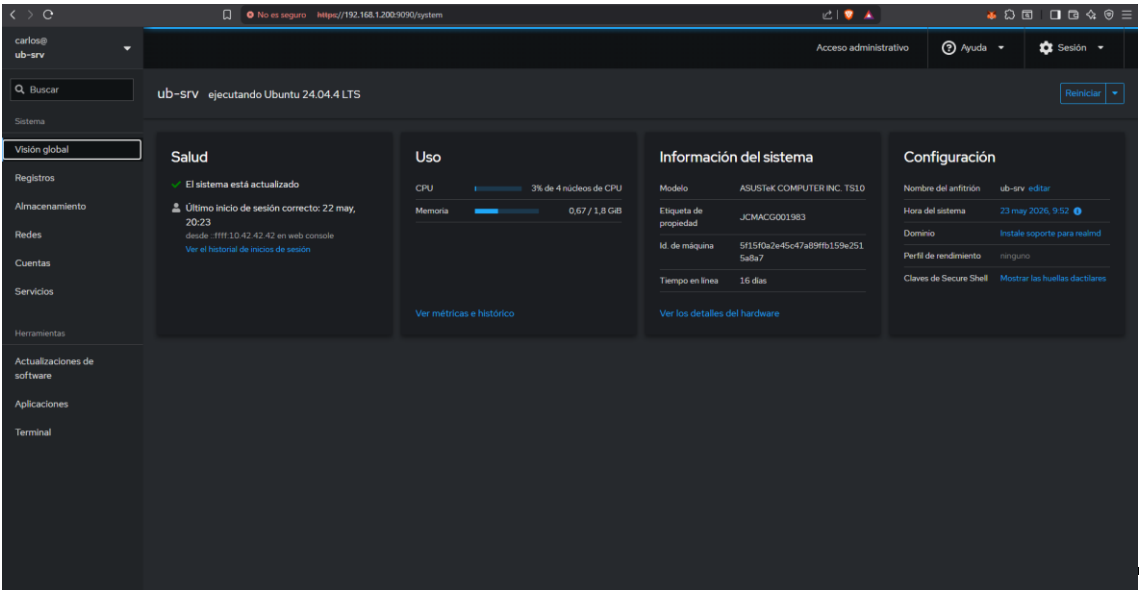
```
# — Volúmenes —
volumes:
  mysql_data:
  mrfinance_root_modules:
  mrfinance_server_modules:
  mrfinance_client_modules:
  certbot_letsencrypt:

# — Red interna —
networks:
  dev_net:
    driver: bridge
```


" " " " " " " " " " " " " " " "

```
nginx.conf
1  server {
2      listen 80;
3      server_name mrfinance.freemyip.com;
4
5      # Redirección automática a HTTPS
6      location / {
7          return 301 https://$host$request_uri;
8      }
9  }
10
11 server {
12     listen 443 ssl;
13     server_name mrfinance.freemyip.com;
14
15     ssl_certificate /etc/letsencrypt/live/mrfinance.freemyip.com/fullchain.pem;
16     ssl_certificate_key /etc/letsencrypt/live/mrfinance.freemyip.com/privkey.pem;
17
18     # Compilación estática de React (Frontend)
19     location / {
20         root /usr/share/nginx/html;
21         index index.html;
22         try_files $uri $uri/ /index.html;
23     }
24
25     # Redirección de peticiones de API al backend
26     # El '/' al final de proxy_pass actúa como el rewrite de Vite, eliminando '/api/' de la petición
27     location /api/ {
28         proxy_pass http://app:8081/;
29         proxy_set_header Host $host;
30         proxy_set_header X-Real-IP $remote_addr;
31         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
32         proxy_set_header X-Forwarded-Proto $scheme;
33     }
34 }
```

```
1  FROM node:24-alpine
2
3  # Build tools needed for native modules (bcrypt requires python3, make, g++)
4  RUN apk add --no-cache python3 make g++
5
6  # Global tools for dev workflow
7  RUN npm install -g nodemon concurrently
8
9  WORKDIR /app
10
11 # Copy the entrypoint
12 COPY entrypoint.sh /entrypoint.sh
13 RUN chmod +x /entrypoint.sh
14
15 EXPOSE 3000 5173 8081 587 443 80
16
17 ENTRYPOINT ["/entrypoint.sh"]
18
```

" " " " " " " " " "

" " " " " " " " " " " " " "

" " " " " "

```
GNU nano 7.2
volumes:
  etc_wireguard:

services:
  wg-easy:
    #environment:
    # Optional:
    # - PORT=51821
    # - HOST=0.0.0.0
    # - INSECURE=false

    image: ghcr.io/wg-easy/wg-easy:15
    container_name: wg-easy
    networks:
      wg:
        ipv4_address: 10.42.42.42
        ipv6_address: fdcc:ad94:bacf:61a3::2a
    volumes:
      - etc_wireguard:/etc/wireguard
      - /lib/modules:/lib/modules:ro
    ports:
      - "51820:51820/udp"
      - "51821:51821/tcp"
    restart: unless-stopped
    cap_add:
      - NET_ADMIN
      - SYS_MODULE
      # - NET_RAW # ⚠️Uncomment if using Podman
    sysctls:
      - net.ipv4.ip_forward=1
      - net.ipv4.conf.all.src_valid_mark=1
      - net.ipv6.conf.all.disable_ipv6=0
      - net.ipv6.conf.all.forwarding=1
      - net.ipv6.conf.default.forwarding=1
    environment:
      - INSECURE=true
    networks:
      wg:
        driver: bridge
        enable_ipv6: true
      ipam:
        driver: default
        config:
          - subnet: 10.42.42.0/24
          - subnet: fdcc:ad94:bacf:61a3::/64
```

" " " " " " " " " " " " " " " "

" " " " " " " " " " " " " " " "

" " " " " " " " " " " " " " " "

" " " " " " " " " " " " " " " "

 **WireGuard**

English

   Administrator

Clients

↓ Sort

+ New

 **movil**
10.8.0.2, fdccad94:bacf61a4:scafe:2
16 hours ago
January 1, 2027

↓ 0 B/s
74.99 MB

↑ 0 B/s
4.12 MB











 **portatil**
10.8.0.3, fdccad94:bacf61a4:scafe:3
January 1, 2027











WireGuard Easy () © 2021-2025 by Emile Nijssen is licensed under AGPL-3.0-only - Donate

" "

“ ” ” ”

" " " " "

Columna1	Columna2	Columna3	Columna4	Columna5	Columna6
" "	" "	" "	" "	" "	" "
" " " " "	" "	" "	" "	" "	" "
" " " "	" "	" "	" "	" "	" "
" " "	" "	" "	" "	" "	" "
" "	" "	" "	" "	" "	" "
" " " "	" "	" "	" "	" "	" "
" "	" "	" "	" "	" "	" "
" " " "	" "	" "	" "	" "	" "
" "	" "	" "	" "	" "	" "

“ “ “ “

11

" " " "

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85															

"10 Horas de formación"	"	"	" "	" "
" " " "	" " " "	" "	" "	" "

" 250 Horas realizando la documentación" " " " " "

"150 Horas codificando la app.

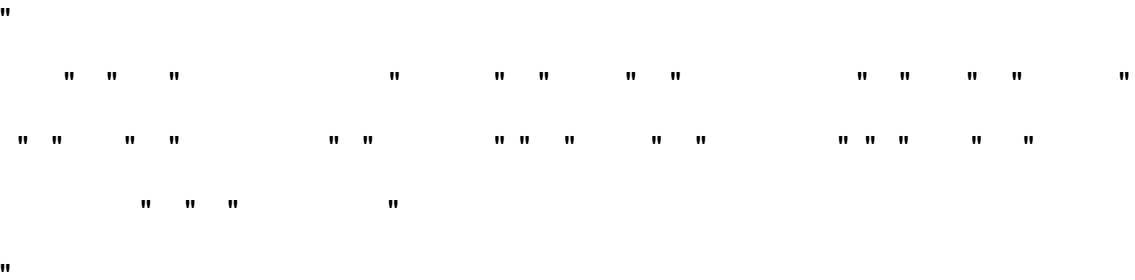
"12 Horas Creando el entorno de desarrollo en Docker.

"5 Horas El despliegue en VPS.

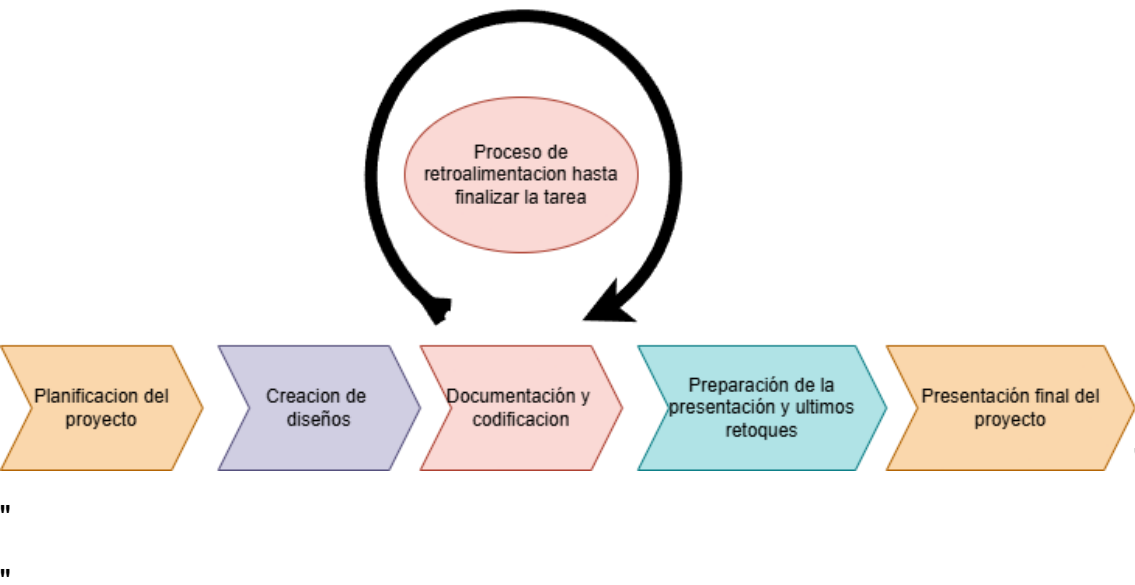
"	"	"				" "327 Horas"	"	"	"	"	"	" "
"	"	"	"	"	"	"	"	"	"Un total de: 4536 euros"			
"	"	"	"	"	"	"	"	"	"	"	"	"
"	"	"	"									

" " "

5.3 Esquema del flujo de trabajo y desarrollo



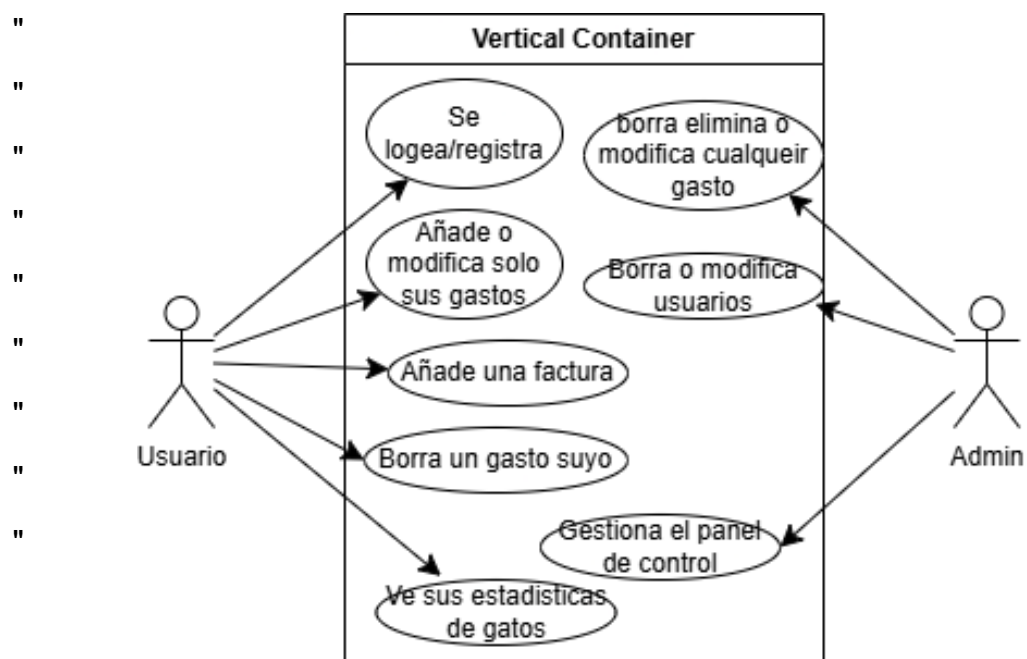
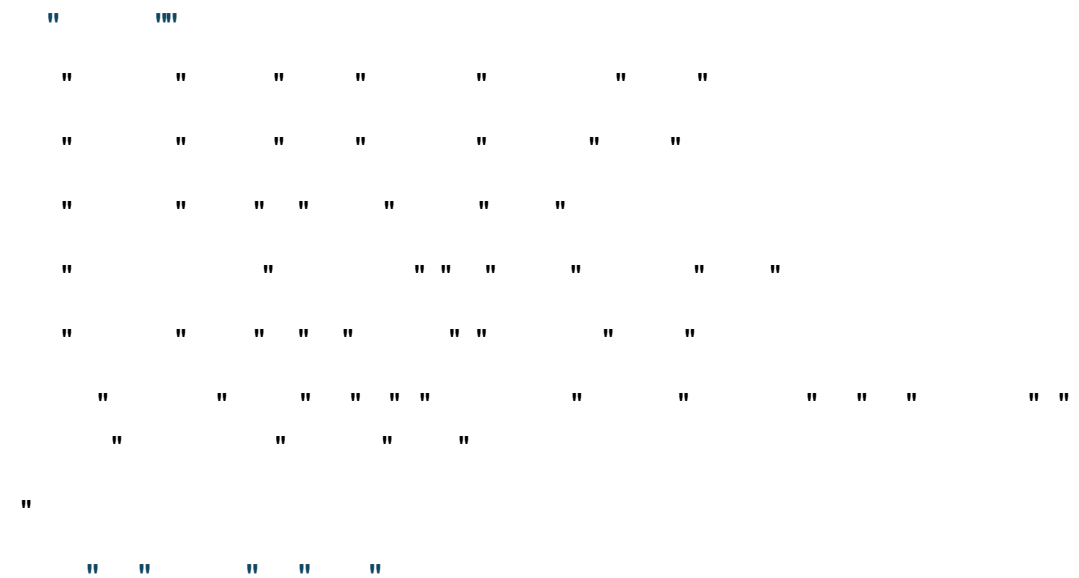
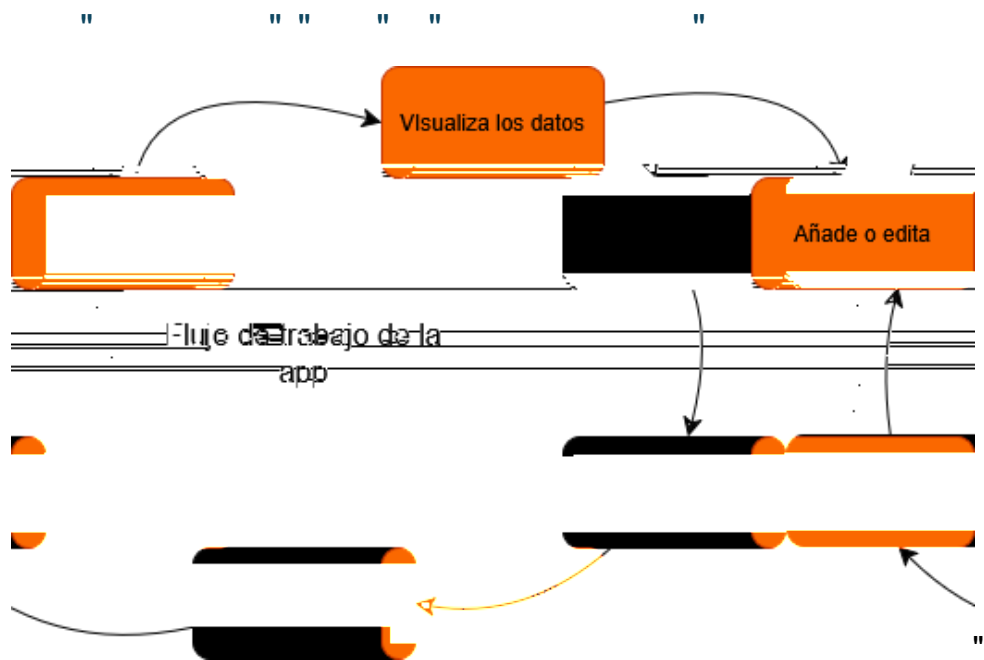
Flujo de Trabajo:



Desarrollo:



[illegible]



"

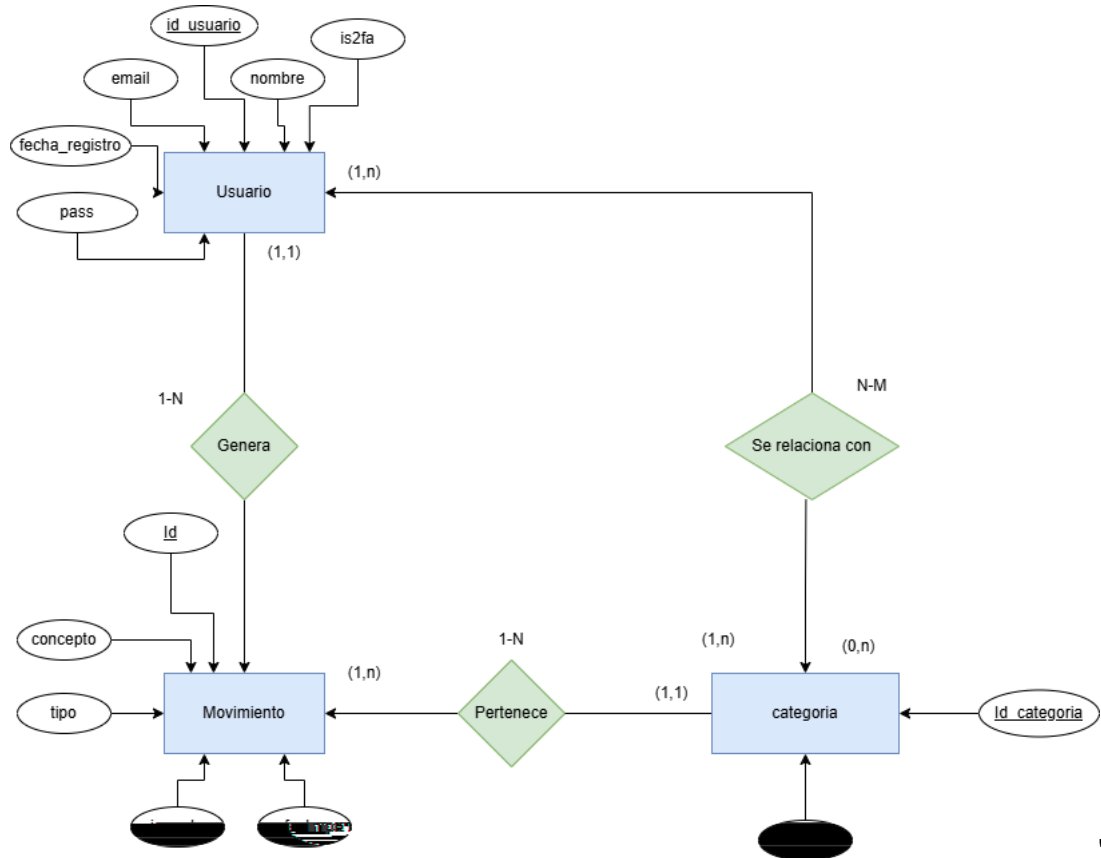
" " " "

"

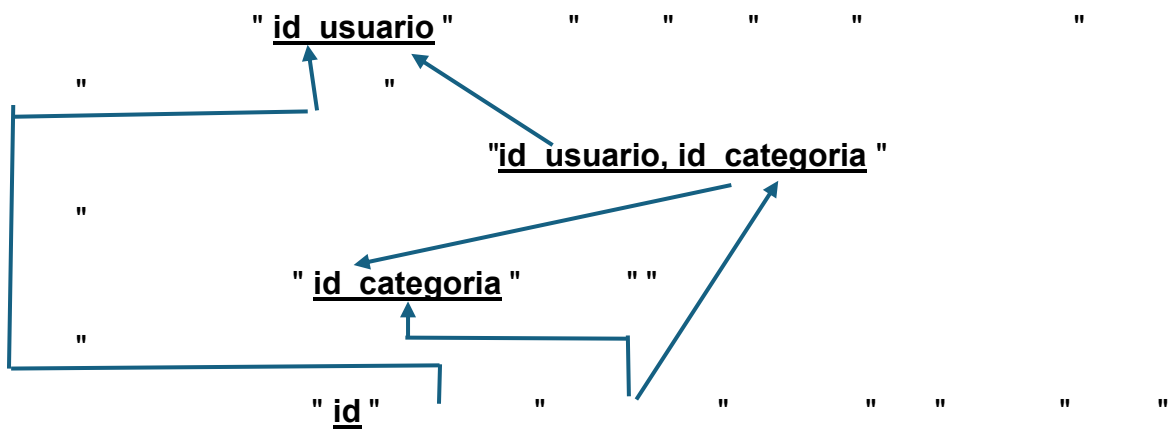
" " " " " "

"

Modelo entidad relación:



Grafo relacional:

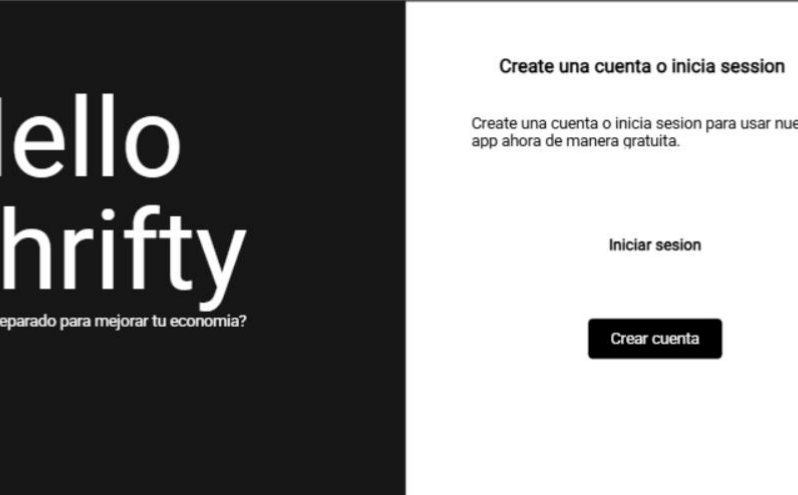


Diccionario de datos de las tablas



二





The screenshot shows a web browser window with the URL "http://127.0.0.1:5500/". The page has a dark left sidebar with the text "Hello Thrifty" and "¿Estas preparado para mejorar tu economía?". The main content area is white and contains the heading "Create una cuenta o inicia session", a paragraph "Create una cuenta o inicia session para usar nuestra app ahora de manera gratuita.", a link "Iniciar session", and a "Crear cuenta" button.

http://127.0.0.1:5500/

Hello Thrifty

¿Estas preparado para mejorar tu economía?

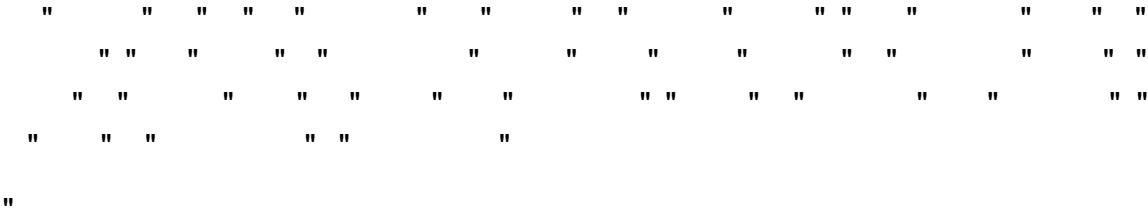
Create una cuenta o inicia session

Create una cuenta o inicia session para usar nuestra app ahora de manera gratuita.

[Iniciar session](#)

[Crear cuenta](#)





```
" " " " " " " " " " " " " " " "
```

```
" " " "
```

```
" " " " " " " " " " " " " "
```

```
" docker compose up arranca 3 servicios" " " " " "
```

```
" " "
```

```
" MySQL" " " " " " " " " " " " " "
```

```
" " " " " " " " " " " " " "
```

```
" "
```

```
" entrypoint.sh" " " " " " " " "
```

```
" " " " " " " " " " " " " "
```

```
" " "
```

```
" El login de MrFinance" " "http://localhost:5173"
```

```
" "
```

"

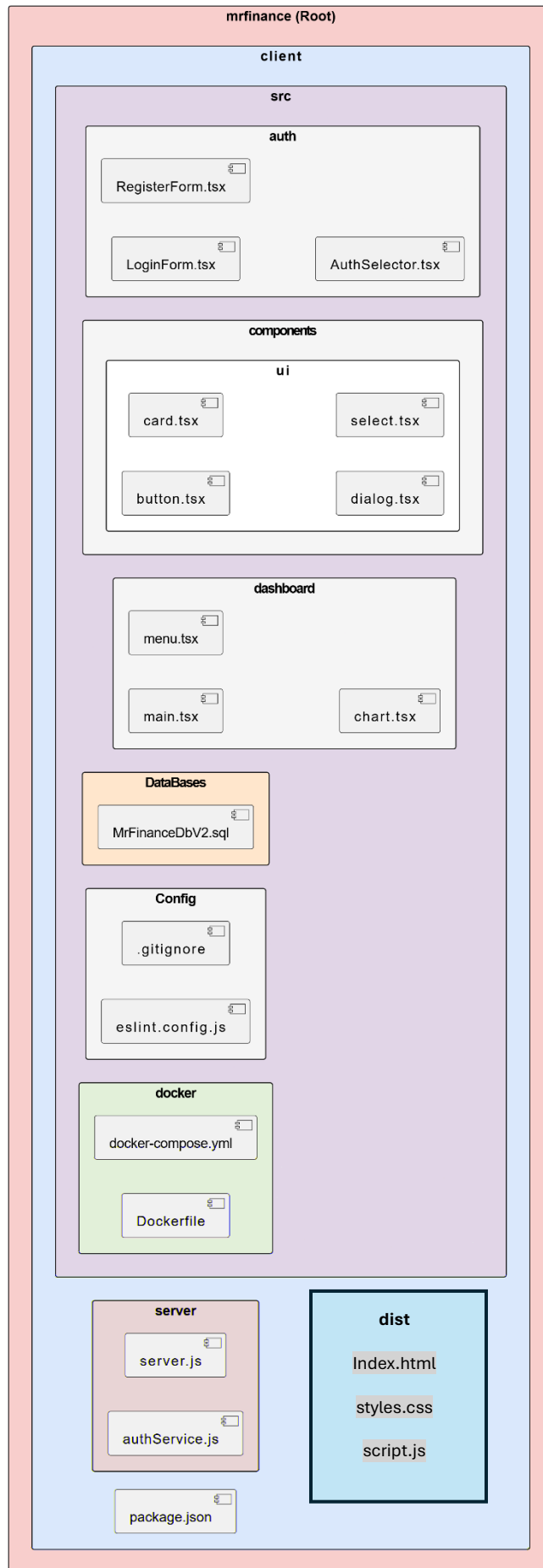
"

"

"

"

"



Podemos ver en el esquema grafico como está estructurado el proyecto, esta es la estructura por defecto en un proyecto vite, algunas características principales de esta estructura son la separación clara del backend con el frontend y mi propia organización de ficheros a la hora de programar los componentes como la separación de la carpeta auth y dashboard, que separan el login de las vistas de cada usuario con sus respectivos datos etc.

Otro detalle es la carpeta components que engloba componentes react de shadcn para luego poderlos usar más adelante en el frontend."

```

1  -- =====
2  -- Base de datos: MrFinanceV2
3  -- Segunda version con triggers para el borrado automatico en vez de on delete cascade
4  -- =====
5
6  DROP DATABASE IF EXISTS MrFinanceV2;
7  CREATE DATABASE IF NOT EXISTS MrFinanceV2 DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
8
9  USE MrFinanceV2;
10
11  -- -----
12  -- Tabla: usuarios
13  -- -----
14  CREATE TABLE usuarios (
15      id          INT          NOT NULL AUTO_INCREMENT,
16      nombre      VARCHAR(100) NOT NULL,
17      email       VARCHAR(120) NOT NULL,
18      pass        VARCHAR(120) NOT NULL,
19      is_2fa      BOOLEAN      NOT NULL DEFAULT FALSE,
20      fecha_registro DATETIME  NOT NULL DEFAULT CURRENT_TIMESTAMP,
21      PRIMARY KEY (id),
22      -- unique key en los email para que no existan usuarios duplicados
23      UNIQUE KEY uq_usuarios_email (email)
24  );
25
26  -- -----
27  -- Tabla: categoria
28  -- -----
29  CREATE TABLE categoria (
30      id          INT          NOT NULL AUTO_INCREMENT,
31      nombre      VARCHAR(100) NOT NULL,
32      id_usuario  INT NOT NULL,
33      FOREIGN KEY (id_usuario) REFERENCES usuarios (id)
34      PRIMARY KEY (id)
35  );
36
37  -- -----
38  -- Tabla: movimientos
39  -- -----
40
41  CREATE TABLE movimientos (
42      id          INT          NOT NULL AUTO_INCREMENT,
43      concepto    VARCHAR(255) NOT NULL,
44      tipo        ENUM('ingreso','gasto') NOT NULL,
45      monto       DECIMAL(10,2) NOT NULL,
46      fecha       DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
47      id_usuario  INT          NOT NULL,
48      id_categoria INT          NOT NULL,
49      PRIMARY KEY (id),
50      -- uso indices para que la bd cargue en cache esos campos y no tarden tanto las consultas
51      INDEX idx_movimientos_usuario (id_usuario),
52      INDEX idx_movimientos_categoria (id_categoria),
53      INDEX idx_movimientos_fecha (fecha),
54      -- foreign keys referentes a los usuarios y categorias
55      FOREIGN KEY (id_usuario) REFERENCES usuarios (id)
56      FOREIGN KEY (id_categoria) REFERENCES categoria (id)
57  );
58
59  -- =====
60  -- Triggers
61  -- =====
62
63  DELIMITER $$
64
65  -- -----
66  -- Trigger: para antes de eliminar un usuario
67  -- Elimina primero sus movimientos para no crear datos sueltos.
68  -- Despues borra las categorias del usuario.
69  -- -----
70
71  CREATE TRIGGER trg_before_delete_usuario
72  BEFORE DELETE ON usuarios
73  FOR EACH ROW
74  BEGIN
75      DELETE FROM movimientos WHERE id_usuario = OLD.id;
76      DELETE FROM categoria WHERE id_usuario = OLD.id;
77  END$$
78
79  -- -----
80  -- Trigger: para antes de eliminar una categoria
81  -- Impide el borrado si existen movimientos con esa categoria,
82  -- devolviendo un mensaje de error si se intenta hacer el delete.
83  -- -----
84
85  CREATE TRIGGER trg_before_delete_categoria
86  BEFORE DELETE ON categoria
87  FOR EACH ROW
88  BEGIN
89      DECLARE v_total INT;
90
91      SELECT COUNT(*) INTO v_total
92      FROM movimientos
93      WHERE id_categoria = OLD.id;
94      IF v_total > 0 THEN

```

"

"

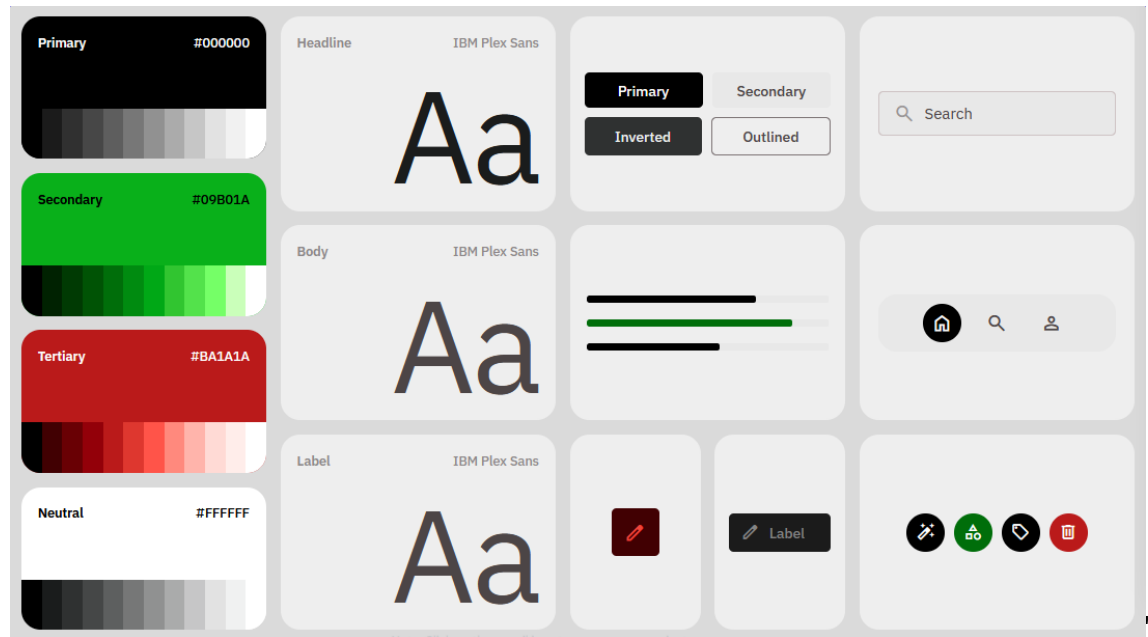
"

"

" " " " " "

" "

"



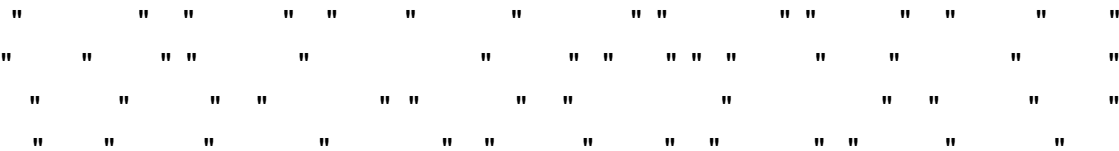
"

"

"

" "

"

[illegible]

Cuestiones de implementación y codificación reseñables

Código node.js (backend)

El backend es un servicio que se arranca en concurrency con el fronted, esto se define el package.json de la raíz, para que arranque a la vez el server y el client,

Se instala con:

```
npm init -y
```

```
npm install -D concurrently
```

Y después se añaden estas líneas al package.json

```
"scripts": {  
  "dev": "concurrently -n SERVER,CLIENT -c blue,green \"npm run dev --prefix server\" \"npm run dev --prefix client\"",  
  "install:all": "npm install --prefix server && npm install --prefix client",  
  "build": "npm run build --prefix client"  
},  
"devDependencies": {  
  "concurrently": "^9.2.1"  
},
```

Luego al hacer npm run dev se ve así:

```
> concurrently -n SERVER,CLIENT -c blue,green "npm run dev --prefix server" "npm run dev --prefix client"  
  
[SERVER]  
[SERVER] > mrfinance-server@1.0.0 dev  
[SERVER] > nodemon server.js  
[SERVER]  
[CLIENT]  
[CLIENT] > mrfinance@0.0.0 dev  
[CLIENT] > vite  
[CLIENT]  
[SERVER] [nodemon] 3.1.14  
[SERVER] [nodemon] to restart at any time, enter `rs`  
[SERVER] [nodemon] watching path(s): *.*  
[SERVER] [nodemon] watching extensions: js,mjs,cjs,json  
[SERVER] [nodemon] starting `node server.js`  
[CLIENT]  
[CLIENT] VITE v8.0.9 ready in 2405 ms  
[CLIENT]  
[CLIENT] → Local: http://localhost:5173/  
[CLIENT] → Network: http://172.21.0.4:5173/  
[SERVER] Servidor escuchando en puerto 8081 ✓  
[SERVER] Conectado a MySQL ✓
```

Luego todo lo que es el código del backend está definido en la carpeta server en el archivo server.js, este archivo es ejecutado por los logs azules que pone server, se puede ver que arranca correctamente por las líneas de Servidor escuchando y conectado a mysql.

A la hora de programar este archivo me he encontrado con el reto de aprender js adaptado a un entorno backend y todo lo que eso conlleva. Estas son algunas capturas de funciones interesantes: Capturas de pantalla del archivo:

" " " " "

```
You, 2 days ago | 2 authors (You and one other)
1 import express from 'express';
2 import mysql from 'mysql2';
3 import cors from 'cors';
4 import bcrypt from 'bcrypt';
5
6 //importo las funciones para el 2fa
7 import { enviarCodigoVerificacion, verificarCodigo } from './authService.js';
8
9 const app = express();
10
11 app.use(cors());
12 app.use(express.json());
13
14 const db = mysql.createConnection({
15   host: "db",
16   user: "user",
17   password: "user",
18   database: "MrFinanceV2",
19   charset: "utf8mb4"
20 });
```

" " "

```
// Logica de login
app.post('/login', (req, res) => {
  const { email, password } = req.body;

  Ctrl+L For Agent
  const sql = "SELECT * FROM usuarios WHERE email = ?";
  db.query(sql, [email], async (err, data) => {
    if (err) {
      console.error("Error en query:", err.message);
      return res.status(500).json({ success: false, message: "Error del servidor" });
    }

    if (data.length === 0) {
      return res.status(401).json({ success: false, message: "Email o contraseña incorrectos" });
    }

    const user = data[0];
    const isBcryptHash = user.pass && user.pass.startsWith('$2');

    let passwordMatch = false;
    if (isBcryptHash) {
      passwordMatch = bcrypt.compareSync(password, user.pass);
    } else {
      passwordMatch = (password === user.pass);
      if (passwordMatch) {
        const salt = bcrypt.genSaltSync(10);
        const hashedPass = bcrypt.hashSync(password, salt);
        db.query("UPDATE usuarios SET pass = ? WHERE email = ?", [hashedPass, user.email]);
        console.log(`Contraseña migrada a bcrypt para: ${user.email}`);
      }
    }

    if (!passwordMatch) {
      return res.status(401).json({ success: false, message: "Email o contraseña incorrectos" });
    }

    // Comprobar si tiene 2FA activo en la BD
    if (user.is_2fa) {
      try {
        await enviarCodigoVerificacion(user.email);
        // No devolvemos los datos del usuario todavía, solo avisamos al frontend
        return res.status(200).json({ success: true, requires2fa: true, email: user.email });
      } catch (error) {
        console.error("Error enviando código 2FA:", error.message);
        return res.status(500).json({ success: false, message: "Error enviando el código de verificación" });
      }
    }

    // Sin 2FA: login directo
    return res.status(200).json({ success: true, user });
  });
});
```

```

1  import transporter from './mailer.js';
2
3  const codigosActivos = new Map();
4
5  async function enviarCodigoVerificacion(emailDestino) { //seactiva cuando se le pasa un mail
6    const codigo = Math.floor(1000 + Math.random() * 9000).toString(); //genera un numero aleatorio de 4 digitos
7    const expiracion = Date.now() + 10 * 60 * 1000; //establece la expiracion en 10 minutos
8
9    codigosActivos.set(emailDestino, { codigo, expiracion }); //guarda el codigo y la expiracion
10
11    const info = await transporter.sendMail({
12      from: `MR Finance <${process.env.GMAIL_USER}>`, //correo del remitente
13      to: emailDestino, //correo del destinatario
14      subject: "Tu codigo de verificacion", //asunto del correo
15      text: `Tu codigo es: ${codigo}\nExpira en 10 minutos.`, //texto del correo
16      html: `
17        <div style="font-family:sans-serif;max-width:400px;margin:auto;padding:24px;
18          border:1px solid #e0e0e0;border-radius:8px;">
19          <h2 style="color:black;">Verificacion en 2 pasos de MR FINANCE</h2>
20          <p>Usa el siguiente codigo para completar tu inicio de sesion:</p>
21          <div style="font-size:36px;font-weight:bold;letter-spacing:8px;
22            text-align:center;color:black;padding:16px 0;">
23            ${codigo}
24          </div>
25          <p style="color:gray;font-size:13px;">
26            <strong color:black>Este codigo expira en 10 minutos</strong>.<br>
27            Si no solicitaste este codigo ignora este mensaje.
28          </p>
29        </div>
30      `;
31    });
32
33    console.log("Codigo enviado. Message ID:", info.messageId);
34    return true;
35  }
36
37  function verificarCodigo(emailDestino, codigoIngresado) { //se activa cuando se le pasa un mail y un codigo
38    const entrada = codigosActivos.get(emailDestino);
39
40    //comprueba que el codigo sea correcto
41    if (!entrada) {
42      return { valido: false, mensaje: "No hay codigo activo para este correo." };
43    }
44
45    if (Date.now() > entrada.expiracion) {
46      codigosActivos.delete(emailDestino);
47      return { valido: false, mensaje: "El codigo ha expirado." };
48    }
49
50    if (entrada.codigo !== codigoIngresado) {
51      return { valido: false, mensaje: "Codigo incorrecto." };
52    }
53
54    codigosActivos.delete(emailDestino); //elimina el codigo
55    return { valido: true, mensaje: "Verificacion exitosa." };
56  }
57
58  export { enviarCodigoVerificacion, verificarCodigo };

```

" " " " " " " " " " " " " " " "

```
1 import nodemailer from "nodemailer";
2 import dotenv from "dotenv";
3 dotenv.config();
4
5 const transporter = nodemailer.createTransport({
6   host: "smtp.gmail.com",
7   port: 587,
8   secure: false, // STARTTLS, Gmail usa 587 con upgrade
9   auth: {
10     user: process.env.GMAIL_USER, //usuario
11     pass: process.env.GMAIL_APP_PASS, //contraseña
12   },
13 });
14
15 //comprueba que el servidor SMTP este listo y no de errores
16 transporter.verify((error) => {
17   if (error) {
18     console.error("Error de conexion SMTP:", error.message);
19   } else {
20     console.log("Servidor SMTP listo para enviar mensajes");
21   }
22 });
23
24 export default transporter;
```

Endpoint que se encarga de verificar los códigos que llegan al server y función para registrarse

"

```
83 // Endpoint para verificar el codigo 2FA
84 app.post('/verify-2fa', (req, res) => {
85   const { email, codigo } = req.body;
86
87   //ni no se le pasa el mail o el codigo da error
88   if (!email || !codigo) {
89     return res.status(400).json({ success: false, message: "Faltan datos" });
90   }
91
92   const resultado = verificarCodigo(email, codigo);
93
94   if (!resultado.valido) {
95     return res.status(401).json({ success: false, message: resultado.mensaje });
96   }
97
98   //Codigo correcto: devuelve los datos del usuario
99   const sql = "SELECT * FROM usuarios WHERE email = ?";
100   db.query(sql, [email], (err, data) => {
101     if (err) {
102       return res.status(500).json({ success: false, message: "Error del servidor" });
103     }
104     return res.status(200).json({ success: true, user: data[0] });
105   });
106 });
107
108 // Logica de registro
109 app.post('/register', (req, res) => {
110   const { email, password, nombre } = req.body;
111
112   // Comprueba que la contraseña sea de minimo 12 caracteres
113   if (!password || password.length < 12) {
114     return res.status(401).json({ success: false, message: "La contraseña es muy corta, tiene que ser de minimo 12 caracteres" });
115   }
116
117   // Primero comprueba si ya existe un usuario con ese email
118   const sqlCheckUser = "SELECT * FROM usuarios WHERE email = ?";
119   db.query(sqlCheckUser, [email], (err, data) => {
120     if (err) {
121       console.error("Error en query:", err.message);
122       return res.status(500).json({ success: false, message: "Error del servidor" });
123     }
124
125     // Si ya existe, devuelve conflicto
126     if (data.length > 0) {
127       return res.status(409).json({ success: false, message: "El usuario ya existe" });
128     }
129
130     // Si no existe, encripta la contraseña y lo inserta
131     const salt = bcrypt.genSaltSync(10);
132     const hashedPass = bcrypt.hashSync(password, salt);
133
134     const sqlInsertUser = "INSERT INTO usuarios (email, pass, nombre) VALUES(?, ?, ?)";
135     db.query(sqlInsertUser, [email, hashedPass, nombre], (err, result) => {
136       if (err) {
137         console.error("Error en query:", err.message);
138         return res.status(500).json({ success: false, message: "Error del servidor" });
139       }
140
141       const user = {
142         id: result.insertId,
143         email: email,
144         pass: hashedPass,
145         nombre: nombre
146       };
147
148       return res.status(200).json({ success: true, user });
149     });
150   });
151 });
152 });
```

" " "

"

```
1 import axios from "axios";
2 import { useState } from "react";
3 import { AuthenticationModal } from "../AuthenticationModal";
4
5 type Props = {
6   onBack: () => void;
7   onSuccess: (user: any) => void;
8 };
9
10 export default function LoginForm({ onBack, onSuccess }: Props) {
11   //toda la logica del login
12   //aquí guarda los datos de email pass...
13   const [email, setEmail] = useState("");
14   // Estado para el modal 2FA
15   const [modal2fa, setModal2fa] = useState(false);
16   const [email2fa, setEmail2fa] = useState("");
17
18   //funcion handleSubmit que se ejecuta al hacer submit en el form de login
19   const handleSubmit = (evento: React.FormEvent) => {
20     //evita que se recargue la pagina
21     evento.preventDefault();
22     setError("");
23
24     //petición axios al backend
25     axios
26       .post("/api/login", { email, password })
27       .then((res) => {
28         if (res.data.success && res.data.requires2fa) {
29           // El servidor ha enviado el código al correo + abrir modal 2FA
30           setEmail2fa(res.data.email);
31           setModal2fa(true);
32         } else if (res.data.success) {
33           // Login directo sin 2FA
34           onSuccess(res.data.user);
35         } else {
36           setError(res.data.message);
37         }
38       })
39       .catch((err) => {
40         console.error(err);
41         setError("No se pudo iniciar sesión por favor vuelve a intentarlo");
42       });
43   };
44 }
```

```
48 // Cancelar 2FA + volver al formulario limpio
49 function handleCancel2fa() {
50   setModal2fa(false);
51   setEmail2fa("");
52   setPassword("");
53   setError("");
54 }
55
56 return (
57   <div className="loginForm">
58     <form onSubmit={handleSubmit}>
59       <h2>Iniciar sesión</h2>
60       {error && <p style={{ color: "red" }}>{error}</p>}
61       <input
62         type="email"
63         placeholder="Email"
64         value={email}
65         //aquí recoge el valor de el input y lo guarda con la función setEmail
66         onChange={(e) => setEmail(e.target.value)}
67       />
68       <input
69         type="password"
70         placeholder="Contraseña"
71         value={password}
72         onChange={(e) => setPassword(e.target.value)}
73       />
74       <button type="submit">Entrar</button>
75       <button type="button" onClick={onBack}>
76         Volver
77       </button>
78     </form>
79   </div>
80
81   /* Modal de verificación 2FA */
82   <AuthenticationModal
83     open={modal2fa}
84     email={email2fa}
85     onSuccess={(user) => {
86       setModal2fa(false);
87       onSuccess(user);
88     }}
89     onCancel={handleCancel2fa}
90   />
91 );
92
93 );
94 }
```

"

Publish repository

GitHub.com

GitHub Enterprise

Name

TFG_2º_DAW_CarlosPilar

Description

Proyecto de fin de grado de Carlos Pilar Sanz

☐ Keep this code private

Publish repository

Cancel

[illegible]

The screenshot shows a web browser window with the URL `http://localhost:5173/`. The page content includes a large "Hello Thriftv" logo on the left and a login/signup form on the right. The form has the heading "Create una cuenta o inicia sesión" and the subheading "¿Estás preparado para mejorar tu economía de manera gratuita?". There are two buttons: "Iniciar sesión" and "Crear cuenta".

On the right side of the browser, the Lighthouse accessibility audit results are displayed. The overall score is 86. The "Contrast" section shows a failure: "Background and foreground colors do not have a sufficient contrast ratio." The "Best Practices" section shows a failure: "Document does not have a main landmark." Below these, there are sections for "Additional things to manually check" and "Passed audits", both of which are currently empty.

[illegible]

The screenshot shows a web browser window with the URL `http://localhost:5173/`. The page displays the 'Thrifty' logo and a sign-up form. A Lighthouse audit is overlaid on the right side of the browser, showing a score of 96 for 'Best Practices'. The audit lists several issues under the 'THINGS TO CONSIDER' and 'HIGHER LEVEL' categories.

11

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

"

Término	Definición
Node.js	Entorno de ejecución de JavaScript en el servidor. Permite ejecutar código JavaScript fuera del navegador web.
Express	Framework web minimalista y flexible para Node.js. Facilita la creación de aplicaciones web y APIs.
MySQL	Sistema de gestión de bases de datos relacional de código abierto. Se utiliza para almacenar y recuperar datos.
Trigger	Procedimiento almacenado que se ejecuta automáticamente en respuesta a un evento específico en una base de datos.
Shadcn/ui	Kit de componentes de interfaz de usuario modular y personalizable. Permite crear interfaces modernas y consistentes.
Tailwind CSS	Framework de diseño de utilidad en CSS. Facilita la creación de interfaces atractivas y responsivas.
Concurrently	Herramienta de línea de comandos para ejecutar múltiples comandos o scripts de forma concurrente.
Multer	Middleware para Node.js que maneja la carga de archivos multipartes (formularios).
Responsive	Característica de una interfaz de usuario que se adapta automáticamente al tamaño de la pantalla del dispositivo.
OTP	Código de Verificación de Un Solo Uso. Código de seguridad temporal generado para autenticación.
VPS	Virtual Private Server. Servidor virtualizado que proporciona recursos dedicados para hosting.
phpMyAdmin	Interfaz web para gestionar MySQL. Permite administrar bases de datos a través de un navegador.
Cockpit	Panel de administración web para monitorear y gestionar servidores Linux.

Método	Endpoint	Descripción	Autenticación
GET	/api/v1/users	Obtiene una lista de todos los usuarios.	Token
POST	/api/v1/users	Crear un nuevo usuario.	Token
PUT	/api/v1/users/{id}	Actualizar un usuario por ID.	Token
DELETE	/api/v1/users/{id}	Eliminar un usuario por ID.	Token
GET	/api/v1/users/{id}	Obtener un usuario por ID.	Token
POST	/api/v1/login	Autenticación de usuario y contraseña.	Token
POST	/api/v1/logout	Desautenticación de usuario.	Token
POST	/api/v1/register	Registro de un nuevo usuario.	Token
POST	/api/v1/reset-password	Restablecer la contraseña de un usuario.	Token
POST	/api/v1/forgot-password	Recuperación de contraseña por correo electrónico.	Token
POST	/api/v1/verify-email	Verificación de correo electrónico.	Token
POST	/api/v1/refresh-token	Actualizar el token de autenticación.	Token
POST	/api/v1/password-change	Cambiar la contraseña de un usuario.	Token

•" **Tabla resumen de pruebas realizadas y resultados**

ID	Resultado obtenido	¿Superada?
CP-01	Usuario creado correctamente con contraseña cifrada	Si
CP-02	Error 401 devuelto con mensaje de contraseña corta	Si
CP-03	Error 409 por email duplicado	Si
CP-04	Acceso directo al dashboard sin 2FA	Si
CP-05	Error 401 con credenciales incorrectas	Si
CP-06	Modal 2FA mostrado y correo enviado	Si
CP-07	Acceso concedido con código válido	Si
CP-08	Acceso denegado con código incorrecto o expirado	Si
CP-09	Categoría creada y mostrada correctamente	Si
CP-10	Categoría vacía o duplicada rechazada	Si
CP-11	Trigger bloquea el borrado de categoría con movimientos	Si
CP-12	Ingreso registrado y gráfico actualizado	Si
CP-13	Gasto registrado y balance actualizado	Si
CP-14	Importe negativo o cero bloqueado en frontend y backend	Si
CP-15	Movimiento editado correctamente	Si
CP-16	Movimiento eliminado de BD y frontend	Si
CP-17	Foto de perfil actualizada correctamente	Si
CP-18	Campo is_2fa actualizado en base de datos	Si
CP-19	Trigger elimina datos del usuario en cascada	Si

•" **Licencias del proyecto**

El proyecto MrFinance utiliza tres niveles de licencia claramente diferenciados:

Código fuente — GNU AGPL v3 Garantiza la libertad de uso, estudio, modificación y distribución del código. Obliga a publicar cualquier mejora, incluso cuando el software se ofrece como servicio web, evitando que terceros privaticen el trabajo.

Documentación — CC BY-NC-SA 4.0 Permite compartir y adaptar la documentación siempre que se cite la autoría, no se haga con fines comerciales y se mantenga la misma licencia en las obras derivadas.

Marca y diseño — Todos los derechos reservados El nombre MrFinance, el logotipo, los diseños visuales y demás elementos de identidad de marca están reservados y no pueden ser utilizados sin autorización expresa del autor.

Los archivos correspondientes se encuentran en la raíz del repositorio bajo los nombres LICENSE, LICENSE-docs y README.md.